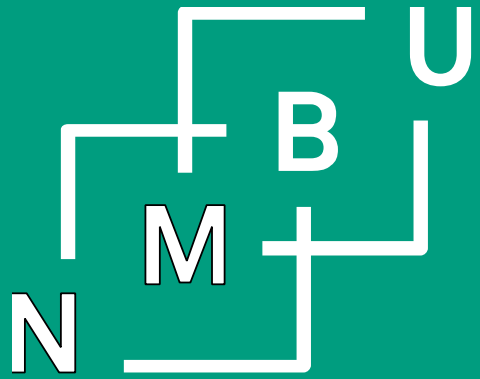


Norwegian University
of Life Sciences

Ensemble learning

Chapter 7 in Python Machine Learning (Third Ed.)

Ensemble learning



https://commons.wikimedia.org/wiki/File:Ensemble_Caf%C3%A9_Zimmermann.jpg

- Imagine asking one expert for a decision—you might get a good answer, or you might not.
- Now imagine asking many experts and combining their opinions into one final decision.

- Ensemble learning: Train **several individual models and then combine their predictions** in some way to get a final prediction that is more accurate and less prone to errors than any individual model.

What we will cover

- What ensemble models are → combining classifications

- How to ensemble models
  - → Majority voting
  - → Bagging – bootstrapping samples
  - → Boosting – weak learners focusing on samples that are hard to classify
    - AdaBoost/XGBoost

- Evaluation and tuning

Resources

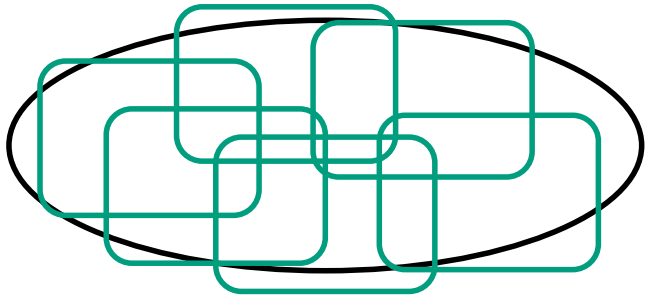
- Python Machine Learning TE, Chapter 7, pages 219-254.
- Jupyter notebook: Chapter 7, part 1 and Chapter 7, part 2

- Ensemble methods in scikit-learn:
  http://scikit-learn.org/stable/modules/ensemble.html

Ensemble Learning

- **Main idea:** Many models in combination can perform better than one -> combine individual classifiers into a meta-classifier.

- **Bagging (Bootstrap aggregating):**
  - Reduce overfit by combining different models built from different subsets of the available training data.



- **Boosting:**
  - Hierarchy of weak learners with different tasks.
  - Combine weak learners into a common, flexible, high performance model (reduces bias).

Analogy - dietary advice

- **Many experts & many opinions**

- **Single learner analogy:**
  - Listen to one expert
  - May be disappointed

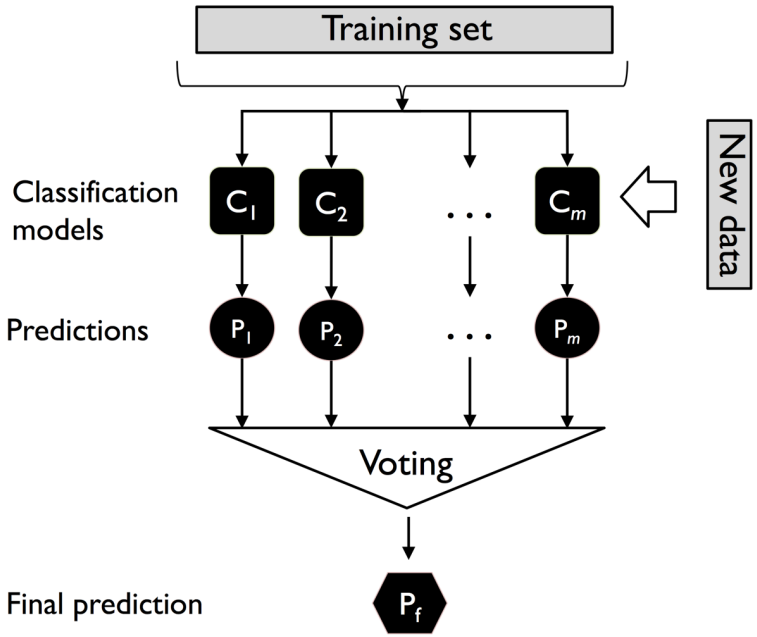
- **Ensemble learning:**
  - Listen to many experts
  - Construct consensus
  - More robust advise
  - Reduce chance of disappointment





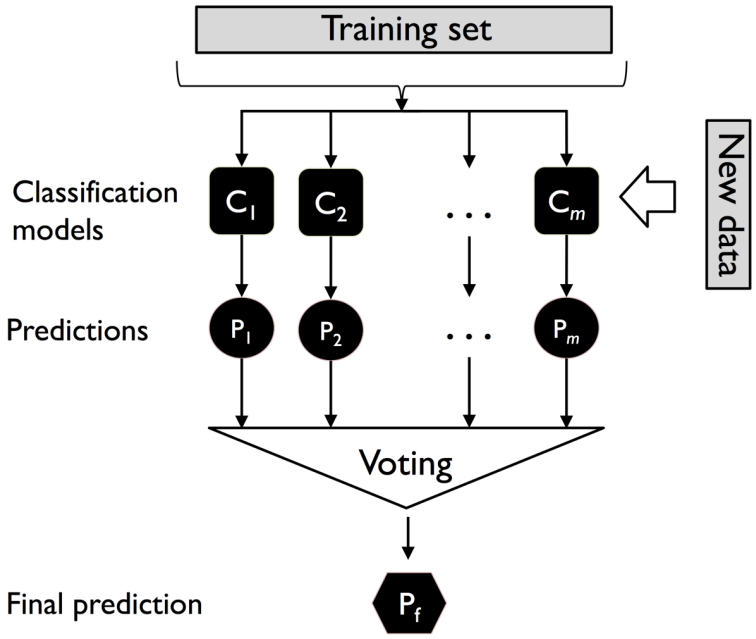
Majority voting

- Train several classifiers

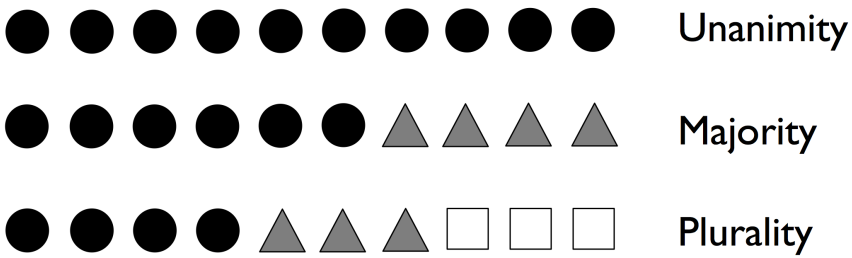


→ Majority voting (binary) simply means that we select the class label that has been predicted by the majority of the classifiers (generalized to multiclass settings, the principle is called plurality voting).

Majority voting



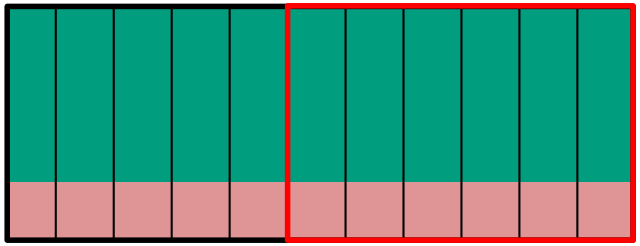
- Train several classifiers



- Choose the class that received the most votes among the classifiers.

$$\hat{y} = mode\{C_1(\boldsymbol{x}), C_2(\boldsymbol{x}), \ldots, C_m(\boldsymbol{x})\}$$

Combinatorial argument



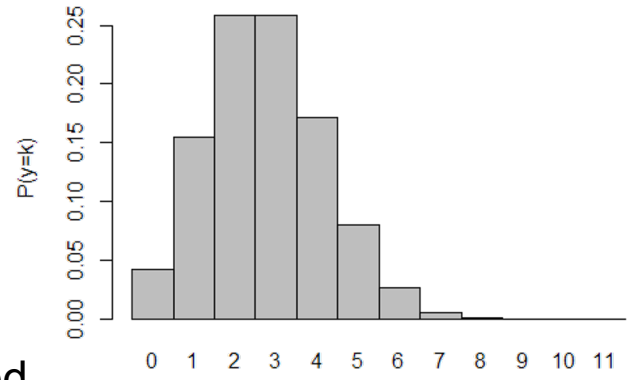
- Assume *n* independent two-class classifiers:
  - equal error rate ε
  - uncorrelated errors
  - *Consider the situation where more than k ≈ n/2* are wrong

$$P(y \geq k) = \sum_{k}^{n} \binom{n}{k} \varepsilon^{k} (1-\varepsilon)^{n-k}$$

- Calculating the chance that 6 or more make a wrong prediction (majority vote fails)

For *n* = 11 and ε = 0.25:

$$P(y \geq 6) = \sum_{k=6}^{11} \binom{11}{k} 0.25^{k} (1-0.25)^{n-k} = 0.034$$



- Note: In practice errors are seldom completely uncorrelated

General majority vote

- **Including weights in the majority voting:**

$$\hat{y}(\boldsymbol{x}) = arg\max_{g}\{\sum_{j=1}^{m} w_j \chi_A\big(C_j(\boldsymbol{x}) = g\big) : g = 1, \dots, G\}$$

  - m: number of classifiers
  - G: number of classes
  - $C_j(x)$: prediction of classifier j on input x
  - $\chi_A(C_j(x) = g)$: 1 if classifier j voted for class g, otherwise 0
  - w_j: weight for classifier j
  - We sum the weights for each class, then pick the class with the **highest total weight**

We look at how many classifiers vote for each class, but instead of just counting votes, we add up their weights (how much we trust each one).

General majority vote - Example

- Example with *m* = 3 classifiers, *G* = 2 classes:
  - w = [0.2, 0.2, 0.6], (weights of the 3 classifiers)
  - C($\boldsymbol{x}$) = [0, 0, 1] (two classifiers vote for “0”, one for “1”)

- Class 0 gets total weight = 0.2 + 0.2 = **0.4**
- Class 1 gets weight = **0.6**

Class 1 wins — it's the final prediction.

=> $\hat{y}(\boldsymbol{x}) = arg\max_{g}\{g = 0{:}\ 0.4, g = 1{:}\ 0.6) = 1\}$

Posterior probabilities

- Change from class membership to probability (p_{gj}) of class membership (predict => `predict_proba`)

$$\hat{y}(\boldsymbol{x}) = arg\max_{g} \sum_{j=1}^{m} w_j \, p_{gj}$$

  - w_j: the **trust level** (weight) assigned to classifier j
  - p_{gj}: the **probability** that classifier j assigns to class g
  - The final prediction is the class g with the **highest total weighted probability** (i.e., the sum of $w_j \cdot p_{gj}$ across all classifiers)

    – (Product of general confidence and class membership probability)

Posterior probabilities - Example

- Change from class membership to probability (p_{gj}) of class membership (predict => `predict_proba`)

$$\hat{y}(\boldsymbol{x}) = arg\max_{g} \sum_{j=1}^{m} w_j \, p_{gj}$$

  - (Product of general confidence and class membership probability)

- Example (same classifier weights: w = [0.2, 0.2, 0.6]):

$$C_1(\boldsymbol{x}) \rightarrow [0.9, 0.1],\ C_2(\boldsymbol{x}) \rightarrow [0.8, 0.2],\ C_3(\boldsymbol{x}) \rightarrow [0.4, 0.6]$$

$$p(g = 0 | \boldsymbol{x}) = 0.2 \times 0.9 + 0.2 \times 0.8 + 0.6 \times 0.4 = 0.58$$

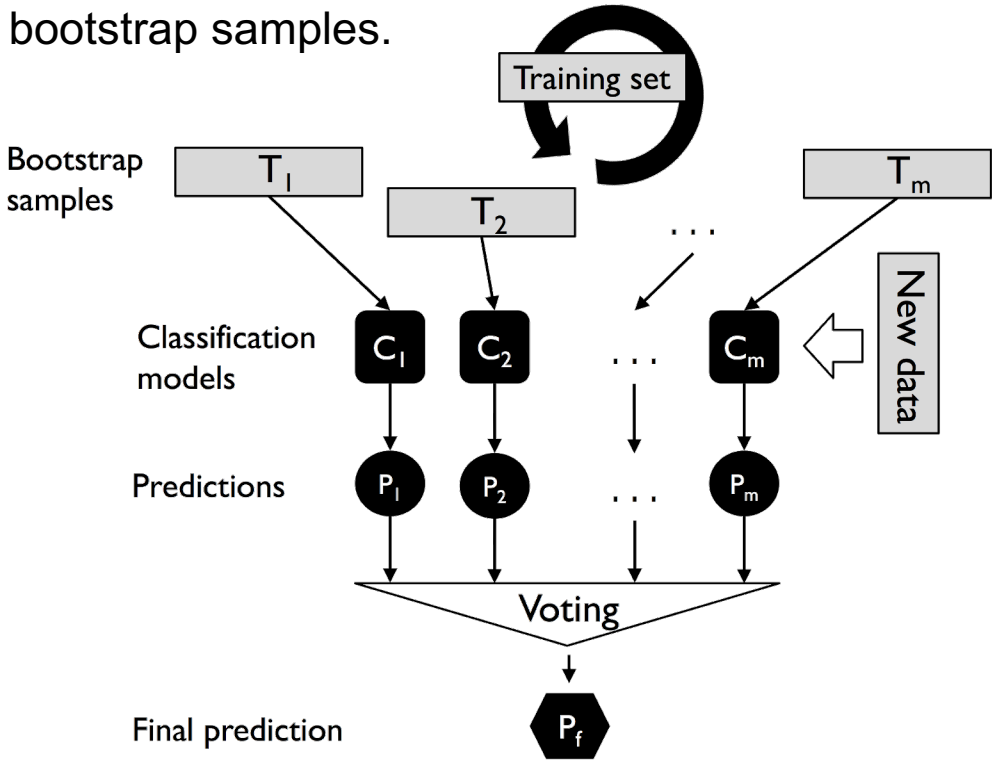
$$p(g = 1 | \boldsymbol{x}) = 0.2 \times 0.1 + 0.2 \times 0.2 + 0.6 \times 0.6 = 0.42$$

$$\hat{y}(\boldsymbol{x}) = arg\max_{g}[\, p(g = 0 | \boldsymbol{x}), p(g = 1 | \boldsymbol{x})] = 0$$

Bagging

Bagging – bootstrap aggregation

- Building an ensemble of classifiers from bootstrap samples.
- Repeatedly sample with replacement from the training data
  - (Replacement causes duplicate observations in the training data)
- Train the classifier on each sample and predict for new data
- Vote for the final prediction



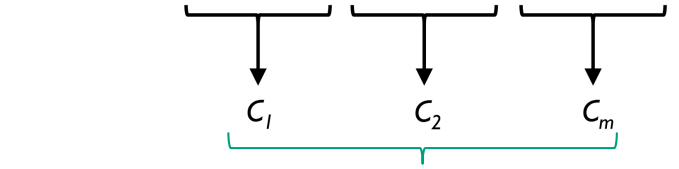
- Note: In regression models, the final prediction is made by averaging the predictions of the individual models -> known as bagging regression.

Bagging “in a nutshell”

- We typically use unpruned **decision trees** as our individual classifiers
  - **Note:** Training accuracy can be forced to 100% with single sample end-nodes.

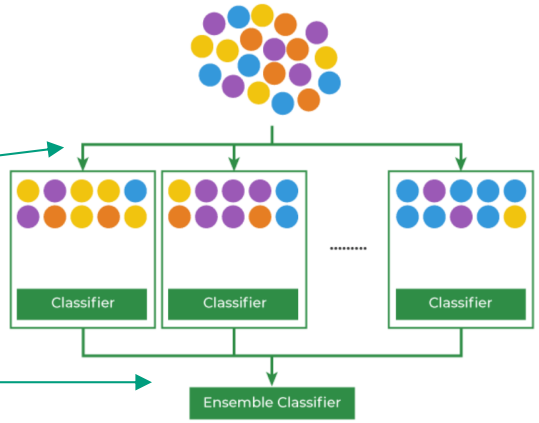
- **Strategy:**
  1. Fit a classifier for each of the *m* bootstrap samples (bootstrap aggregation)
  2. Apply all the *m* classifiers for prediction
  3. Make the final (ensemble) prediction by Majority voting based on the *m* individual classifiers

Sample indices	Bagging round 1	Bagging round 2	…
1	2	7	…
2	2	3	…
3	1	2	…
4	3	1	…
5	7	1	…
6	2	7	…
7	4	7	…



Bagging Classifier

- Basic steps:
  1. Bootstrap sampling
  2. Base model training (individually)
  3. Aggregation/Majority voting



- `from sklearn.ensemble import BaggingClassifier`

Jupyter: Chapter 7, part 2

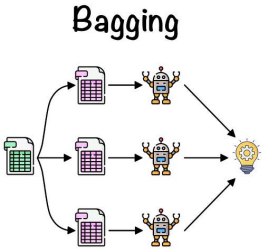
*https://www.geeksforgeeks.org/ml-bagging-classifier/

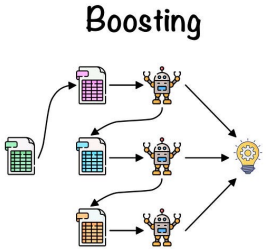
Finding weights for majority voting

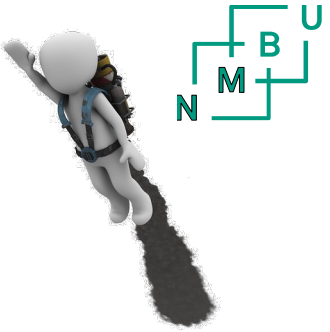
- **Validation set accuracy / Performance:**
  - Evaluate each base model individually on a held-out validation set. Assign higher weights to models that achieve higher accuracy, F1-score, or log-loss.
- **Grid Search / Random Search:**
  - Treat the model weights as hyperparameters, and use a grid search or random search to find the combination of weights that minimizes the validation error or maximizes a chosen metric.
- **Log-odds / Error-rate formulas:**
  - The weights are derived mathematically from the error rate of the classifiers.
  - In AdaBoost (next topic in class): Better models (lower error) receive an exponentially larger positive weight.

Boosting

Boosting “in a nutshell”







- Very simple individual classifiers (weak learners), usually a decision tree stump (a tree with only one split).

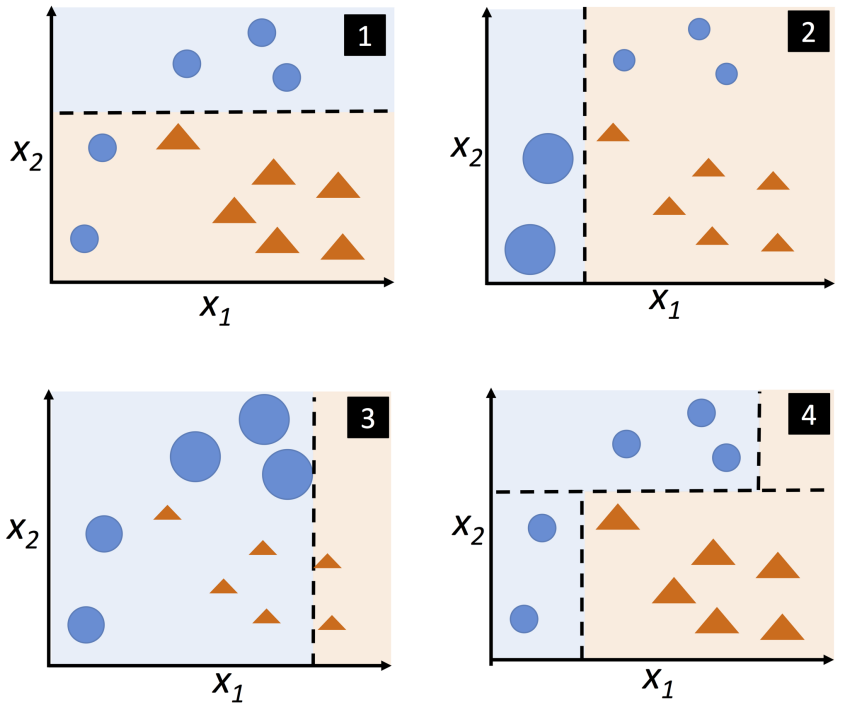
- Focus on the samples that are hard to classify:
  - Re-learning based on up-weighting of previous misclassifications.

- Many different boosting algorithms exist, e.g., AdaBoost, LogitBoost, BrownBoost, LPBoost, XGBoost, …

The AdaBoost

- Complete training set every time
  - Repeated reweighting
  - Learn from previous mistakes

1. First classification
2. Increase weights on misclassified samples, decrease on the correct ones.
3. Repeat, usually several times
4. Combine the classifiers obtained in all the previous steps by weighted majority voting.



AdaBoost, detailed description

1. Set the weight vector $\mathbf{w}$ to uniform weights, where $\sum_i w_i = 1$.
2. For j in m boosting rounds, do the following:
   a. Train a weighted weak learner: $C_j = \text{train}(\boldsymbol{X}, \boldsymbol{y}, \boldsymbol{w})$.

The first time all the sample weights in $\boldsymbol{w}$ are equal.

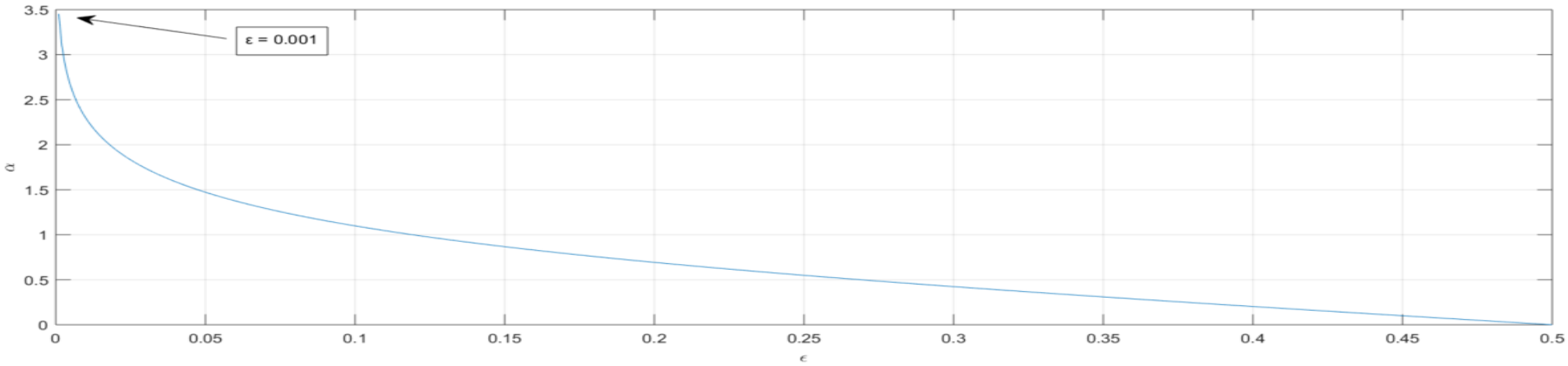
In later iterations the weights are adjusted to emphasize samples that seem hard to classify correctly.

AdaBoost, detailed description

1. Set the weight vector $\mathbf{w}$ to uniform weights, where $\sum_i w_i = 1$.
2. For j in m boosting rounds, do the following:
   a. Train a weighted weak learner: $C_j = \text{train}(\boldsymbol{X}, \boldsymbol{y}, \boldsymbol{w})$.
   b. Predict class labels: $\hat{y} = \text{predict}(C_j, \boldsymbol{X})$.
   c. Compute weighted error rate: $\varepsilon = \boldsymbol{w} \cdot (\hat{\boldsymbol{y}} \neq \boldsymbol{y})$.

Use the sample weights ***w*** to emphasize errors from hard to classify samples

AdaBoost, detailed description



c. Compute weighted error rate: $\varepsilon = \boldsymbol{w} \cdot \left(\hat{\boldsymbol{y}} \neq \boldsymbol{y} \right)$.

d. Compute coefficient: $\alpha_j = 0.5 \log \frac{1-\varepsilon}{\varepsilon}$.

The coefficient α_j quantifies our confidence in the current round of classifications.

It will also be used in the subsequent updating of the sample weights ($\boldsymbol{w}$) (emphasizing samples that are hard to classify).

AdaBoost, detailed description

1. Set the weight vector $\mathbf{w}$ to uniform weights, where $\sum_i w_i = 1$.
2. For j in m boosting rounds, do the following:
   a. Train a weighted weak learner: $C_j = \text{train}(\boldsymbol{X}, \boldsymbol{y}, \boldsymbol{w})$.
   b. Predict class labels: $\hat{y} = \text{predict}(C_j, \boldsymbol{X})$.
   c. Compute weighted error rate: $\varepsilon = \boldsymbol{w} \cdot (\hat{\boldsymbol{y}} \neq \boldsymbol{y})$.
   d. Compute coefficient: $\alpha_j = 0.5 \log \frac{1-\varepsilon}{\varepsilon}$.
   e. Update weights: $\boldsymbol{w} := \boldsymbol{w} \times \exp(-\alpha_j \times \hat{\boldsymbol{y}} \times \boldsymbol{y})$.

Next round's weights are an adjustment of this round's weights according to the confidence in this round and correct/incorrect classification (**y_i = +/- 1**),
-> an element-wise product resulting in a vector.

Note: In the notation below, element-wise multiplication is represented by the cross symbol (×) and the dot-product between two vectors by a dot symbol (·), see more details on pages 250-54:

AdaBoost, detailed description

1. Set the weight vector **w** to uniform weights, where $\sum_i w_i = 1$.
2. For j in m boosting rounds, do the following:
   a. Train a weighted weak learner: $C_j = \text{train}(\boldsymbol{X}, \boldsymbol{y}, \boldsymbol{w})$.
   b. Predict class labels: $\hat{y} = \text{predict}(C_j, \boldsymbol{X})$.
   c. Compute weighted error rate: $\varepsilon = \boldsymbol{w} \cdot (\hat{\boldsymbol{y}} \neq \boldsymbol{y})$.
   d. Compute coefficient: $\alpha_j = 0.5 \log \frac{1-\varepsilon}{\varepsilon}$.
   e. Update weights: $\boldsymbol{w} := \boldsymbol{w} \times \exp(-\alpha_j \times \hat{\boldsymbol{y}} \times \boldsymbol{y})$.
   f. Normalize weights to sum to 1: $\boldsymbol{w} := \boldsymbol{w} / \sum_i w_i$.
3. Compute the final prediction: $\hat{\boldsymbol{y}} = \left(\sum_{j=1}^{m} (\boldsymbol{\alpha}_j \times \text{predict}(C_j, \boldsymbol{X})) > 0\right)$.

Adaboost - Example

x1	x2	x3	Y	Sample Weight
T	T	F	T	1/6
T	T	F	F	1/6
T	F	F	T	1/6
T	T	T	F	1/6
F	T	F	F	1/6
T	F	F	T	1/6

Assume first row was incorrectly classified for weak classifier 1 (stump).

$$\alpha = \frac{1}{2}\log\left(\frac{1-\frac{1}{6}}{\frac{1}{6}}\right) = 0.35$$

Calculate the new sample weights for the next stump

x1	x2	x3	Y	New Sample Weight
T	T	F	T	0.24
T	T	F	F	0.12
T	F	F	T	0.12
T	T	T	F	0.12
F	T	F	F	0.12
T	F	F	T	0.12

Total: 0.84

correct samples = $\frac{\exp(-0.35)}{6} \approx 0.12$

incorrect sample = $\frac{\exp(0.35)}{6} \approx 0.24$

Source: https://towardsdatascience.com/a-mathematical-explanation-of-adaboost-4b0c20ce4382

Adaboost - Example

x1	x2	x3	Y	Sample Weight
T	T	F	T	1/6
T	T	F	F	1/6
T	F	F	T	1/6
T	T	T	F	1/6
F	T	F	F	1/6
T	F	F	T	1/6

Assume first row was incorrectly classified for weak classifier 1 (stump).

$$\alpha = \frac{1}{2}\log\left(\frac{1-\frac{1}{6}}{\frac{1}{6}}\right) = 0.35$$

Need to divide each weight by 0.84

x1	x2	x3	Y	New Sample Weight	Normalized Sample Weights
T	T	F	T	0.24	0.29
T	T	F	F	0.12	0.14
T	F	F	T	0.12	0.14
T	T	T	F	0.12	0.14
F	T	F	F	0.12	0.14
T	F	F	T	0.12	0.14

Total: 0.84

Normalized Weights

incorrect sample = $\frac{0.24}{0.84} = 0.29$

correct samples = $\frac{0.12}{0.84} = 0.14$

Source: https://towardsdatascience.com/a-mathematical-explanation-of-adaboost-4b0c20ce4382

Adaboost - Example

x1	x2	x3	Y	Sample Weight
T	T	F	T	1/6
T	T	F	F	1/6
T	F	F	T	1/6
T	T	T	F	1/6
F	T	F	F	1/6
T	F	F	T	1/6

Assume first row was incorrectly classified for weak classifier 1 (stump).

$$\alpha = \frac{1}{2}\log\left(\frac{1-\frac{1}{6}}{\frac{1}{6}}\right) = 0.35$$

Need to divide each weight by 0.84

x1	x2	x3	Y	New Sample Weight	Normalized Sample Weights
T	T	F	T	0.24	0.29
T	T	F	F	0.12	0.14
T	F	F	T	0.12	0.14
T	T	T	F	0.12	0.14
F	T	F	F	0.12	0.14
T	F	F	T	0.12	0.14

Total: 0.84

Normalized Weights

incorrect sample = $\frac{0.24}{0.84} = 0.29$

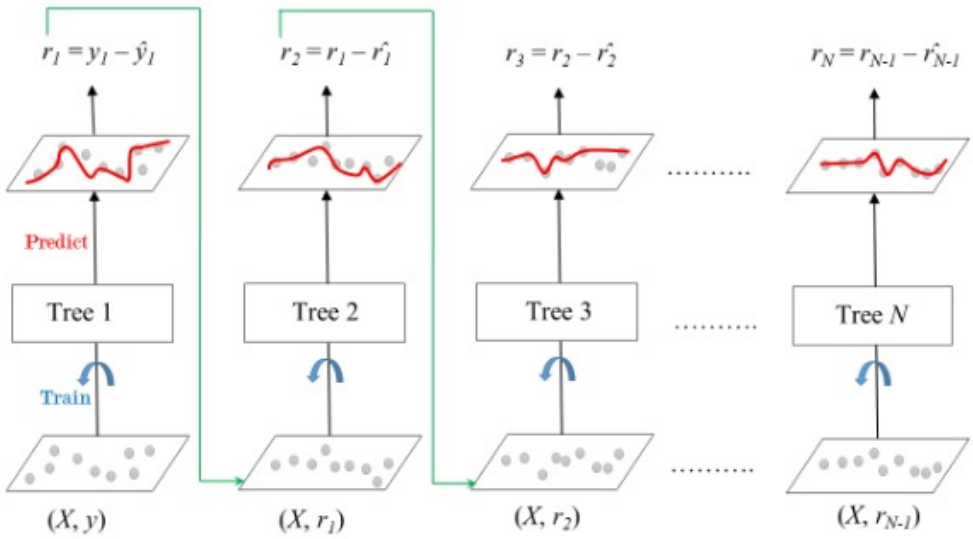
correct samples = $\frac{0.12}{0.84} = 0.14$

Repeat the process *n* times

Source: https://towardsdatascience.com/a-mathematical-explanation-of-adaboost-4b0c20ce4382

Gradient Boosting

1. Initialize a base model
2. Repeat
   i. Compute residuals
   ii. Build a weak learner on the residuals
   iii. Compute the gradients of the loss
   iv. Update the model
3. Make the final prediction



Step-by-step walk through* (regression)

Step 1: Make an initial prediction

- Dataset

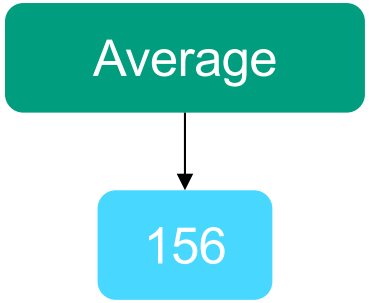
Age	Category	Weight (kg)	Price (USD)
25	Electronics	2.5	123.45
34	Clothing	1.3	56.78
42	Electronics	5.0	345.67
19	Homeware	3.2	98.01

- Loss function (MSE)

$$\frac{1}{2}(Predicted - Observed)^2$$

- Initial prediction (for the target *Y* (price))

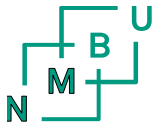
$$\frac{123+56+345+98}{4} = 156$$



**https://www.datacamp.com/tutorial/guide-to-the-gradient-boosting-algorithm*

Step-by-step walk through* (regression)

NOTE: For classification problems the log-loss is used, and the residuals become the difference between the class label and the probability of the positive class ($y_i - p_i$).



Step 2: Calculate the pseudo-residuals

- Dataset

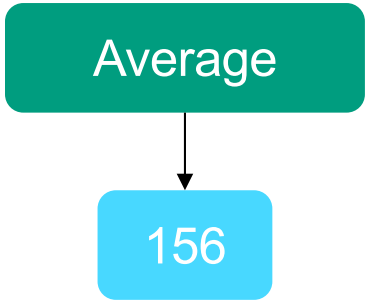
Age	Category	Weight (kg)	Price (USD)	Pseudo residuals
25	Electronics	2.5	123.45	-32.55
34	Clothing	1.3	56.78	-99.22
42	Electronics	5.0	345.67	189.45
19	Homeware	3.2	98.01	-58.01

- Loss function (MSE)

$$\frac{1}{2}(Predicted - Observed)^2$$

- Initial prediction (for the target *Y* (price))

$$\frac{123+56+345+98}{4} = 156$$



Residuals (errors) = Actual price − 156 (average)

Since we use MSE loss, the gradient (direction of steepest improvement) is just the difference between the actual and the predicted values.

*https://www.datacamp.com/tutorial/guide-to-the-gradient-boosting-algorithm

Step-by-step walk through* (regression)

Step 3: Build a weak learner and make a new prediction – a decision tree

- Dataset

Age	Category	Weight (kg)	Price (USD)	Pseudo residuals
25	Electronics	2.5	123.45	-32.55
34	Clothing	1.3	56.78	-99.22
42	Electronics	5.0	345.67	189.45
19	Homeware	3.2	98.01	-58.01

*https://www.datacamp.com/tutorial/guide-to-the-gradient-boosting-algorithm

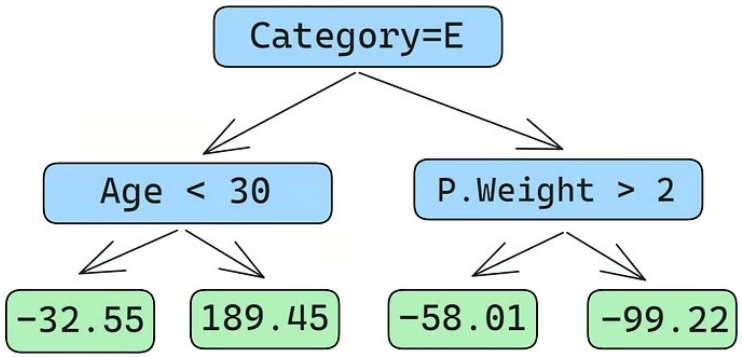
Step-by-step walk through* (regression)

Step 3: Build a weak learner and make a new prediction – a decision tree

- Dataset

Age	Category	Weight (kg)	Price (USD)	Pseudo residuals
25	Electronics	2.5	123.45	-32.55
34	Clothing	1.3	56.78	-99.22
42	Electronics	5.0	345.67	189.45
19	Homeware	3.2	98.01	-58.01

Tree 1



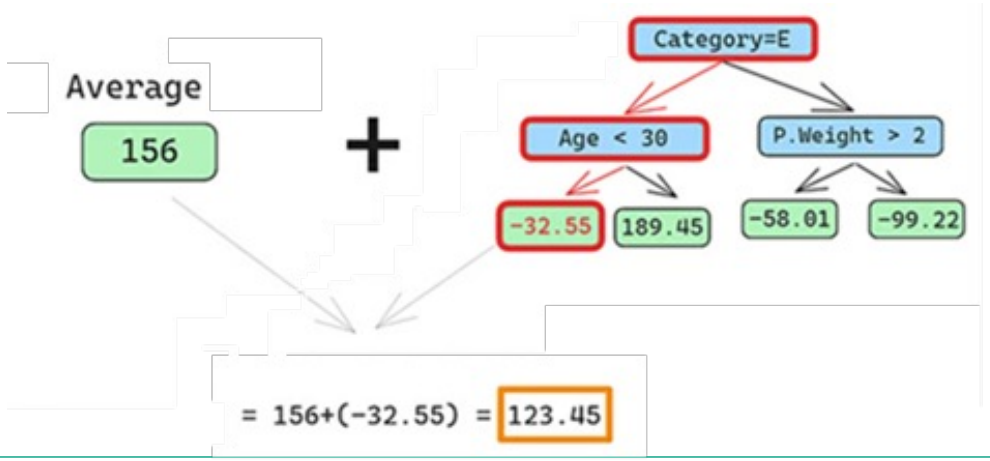
*https://www.datacamp.com/tutorial/guide-to-the-gradient-boosting-algorithm

Step-by-step walk through* (without learning rate)

Step 3: Build a weak learner and make a new prediction – sample 1

- Dataset

Age	Category	Weight (kg)	Price (USD)	Pseudo residuals
25	Electronics	2.5	123.45	-32.55
34	Clothing	1.3	56.78	-99.22
42	Electronics	5.0	345.67	189.45
19	Homeware	3.2	98.01	-58.01



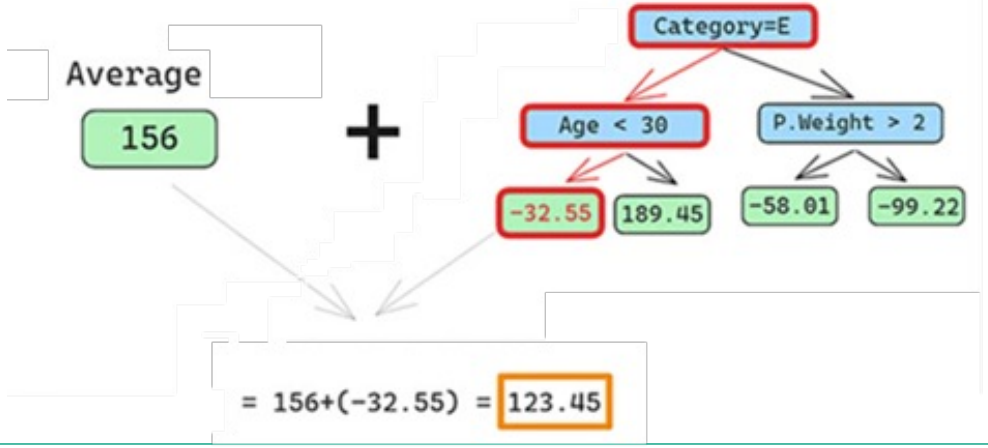
*https://www.datacamp.com/tutorial/guide-to-the-gradient-boosting-algorithm

Step-by-step walk through* (without learning rate)

Step 3: Build a weak learner and make a new prediction – sample 1

- Dataset

Age	Category	Weight (kg)	Price (USD)	Pseudo residuals
25	Electronics	2.5	123.45	-32.55
34	Clothing	1.3	56.78	-99.22
42	Electronics	5.0	345.67	189.45
19	Homeware	3.2	98.01	-58.01



- Adding this to the base prediction gives a perfect match
- It is too perfect on the training data (overfitting)
- To prevent overfitting, we use a learning rate (a small multiplier)
- With a learning rate of 0.1, the update is smaller (the model improves gradually)

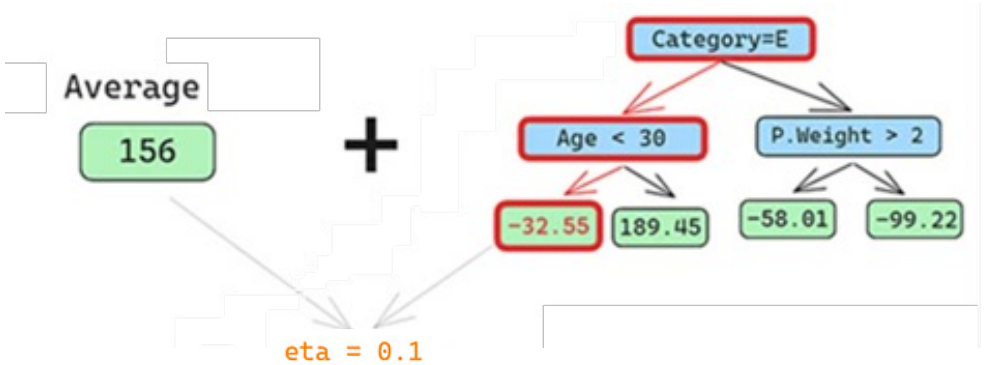
*https://www.datacamp.com/tutorial/guide-to-the-gradient-boosting-algorithm

Step-by-step walk through* (WITH learning rate)

Step 3: Build a weak learner and make a new prediction – sample 1

- Dataset

Age	Category	Weight (kg)	Price (USD)	Pseudo residuals
25	Electronics	2.5	123.45	-32.55
34	Clothing	1.3	56.78	-99.22
42	Electronics	5.0	345.67	189.45
19	Homeware	3.2	98.01	-58.01



$$\text{New prediction} = 156 + 0.1 \times (-32.55) = 156 - 3.255 = \boxed{152.745}$$

New Prediction after tree 1
152.745

*https://www.datacamp.com/tutorial/guide-to-the-gradient-boosting-algorithm

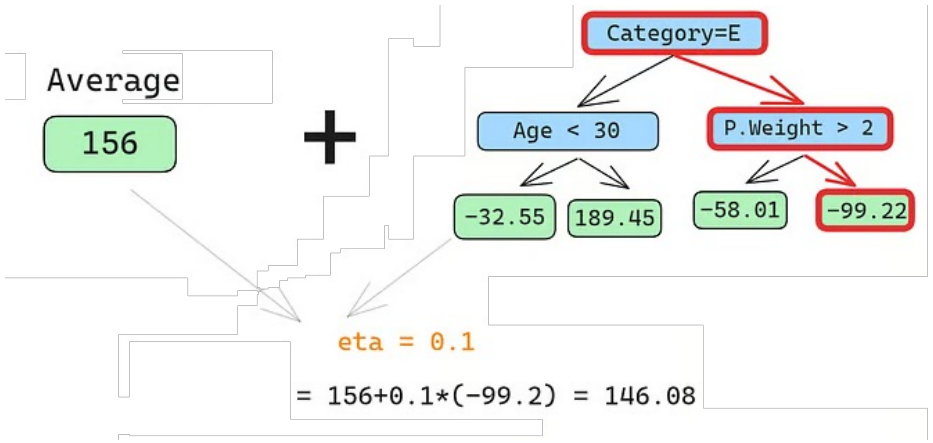
Step-by-step walk through* (WITH learning rate)

Step 3: Build a weak learner and make a new prediction – sample 2

- Dataset

Age	Category	Weight (kg)	Price (USD)
25	Electronics	2.5	123.45
34	Clothing	1.3	56.78
42	Electronics	5.0	345.67
19	Homeware	3.2	98.01

Pseudo residuals
-32.55
-99.22
189.45
-58.01



New Prediction after tree 1
152.745
146.08

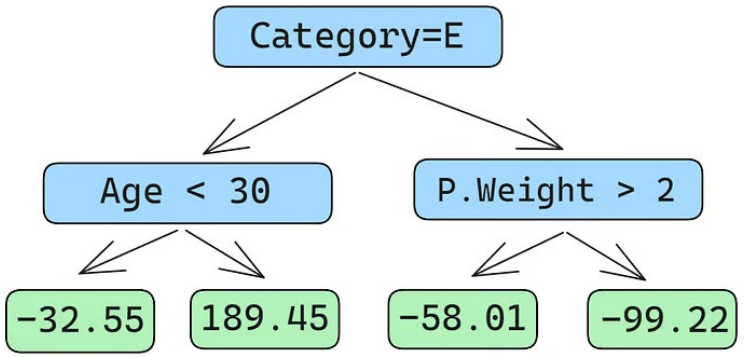
*https://www.datacamp.com/tutorial/guide-to-the-gradient-boosting-algorithm

Step-by-step walk through* (WITH learning rate)

Step 3: Build a weak learner and make a new prediction – all samples

- Dataset

Age	Category	Weight (kg)	Price (USD)	Pseudo residuals
25	Electronics	2.5	123.45	-32.55
34	Clothing	1.3	56.78	-99.22
42	Electronics	5.0	345.67	189.45
19	Homeware	3.2	98.01	-58.01



New Prediction after tree 1
152.745
146.08
174.945
150.199

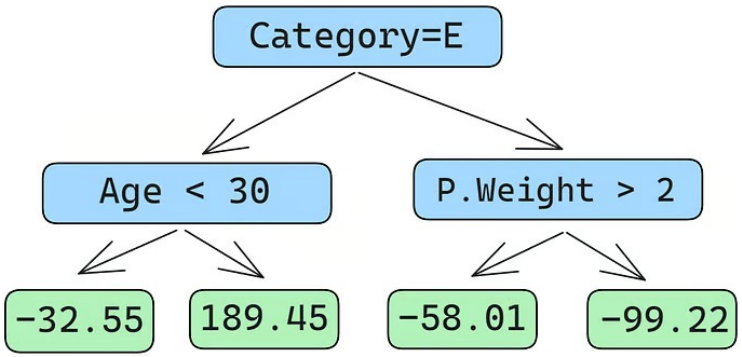
*https://www.datacamp.com/tutorial/guide-to-the-gradient-boosting-algorithm

Step-by-step walk through* (WITH learning rate)

Step 3: Build a weak learner and make a new prediction – all samples

- Dataset

Age	Category	Weight (kg)	Price (USD)	Pseudo residuals
25	Electronics	2.5	123.45	-32.55
34	Clothing	1.3	56.78	-99.22
42	Electronics	5.0	345.67	189.45
19	Homeware	3.2	98.01	-58.01



Residuals after tree 1	New Prediction after tree 1
123.45 − 152.745 = **-29.295**	152.745
56.78 − 146.078 = **-89.298**	146.08
345.67 − 174.945 = **170.725**	174.945
98.01 − 150.199 = **-52.189**	150.199

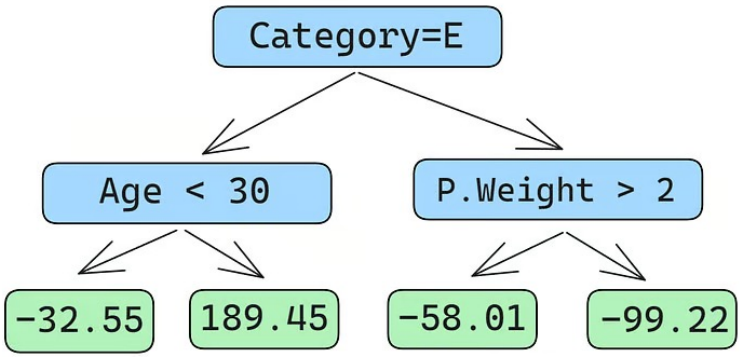
*https://www.datacamp.com/tutorial/guide-to-the-gradient-boosting-algorithm

Step-by-step walk through* (WITH learning rate)

Step 3: Build a weak learner and make a new prediction – all samples

- Dataset

Age	Category	Weight (kg)	Price (USD)	Pseudo residuals
25	Electronics	2.5	123.45	-32.55
34	Clothing	1.3	56.78	-99.22
42	Electronics	5.0	345.67	189.45
19	Homeware	3.2	98.01	-58.01



Residuals after tree 1	New Prediction after tree 1
123.45 − 152.745 = **-29.295**	152.745
56.78 − 146.078 = **-89.298**	146.08
345.67 − 174.945 = **170.725**	174.945
98.01 − 150.199 = **-52.189**	150.199

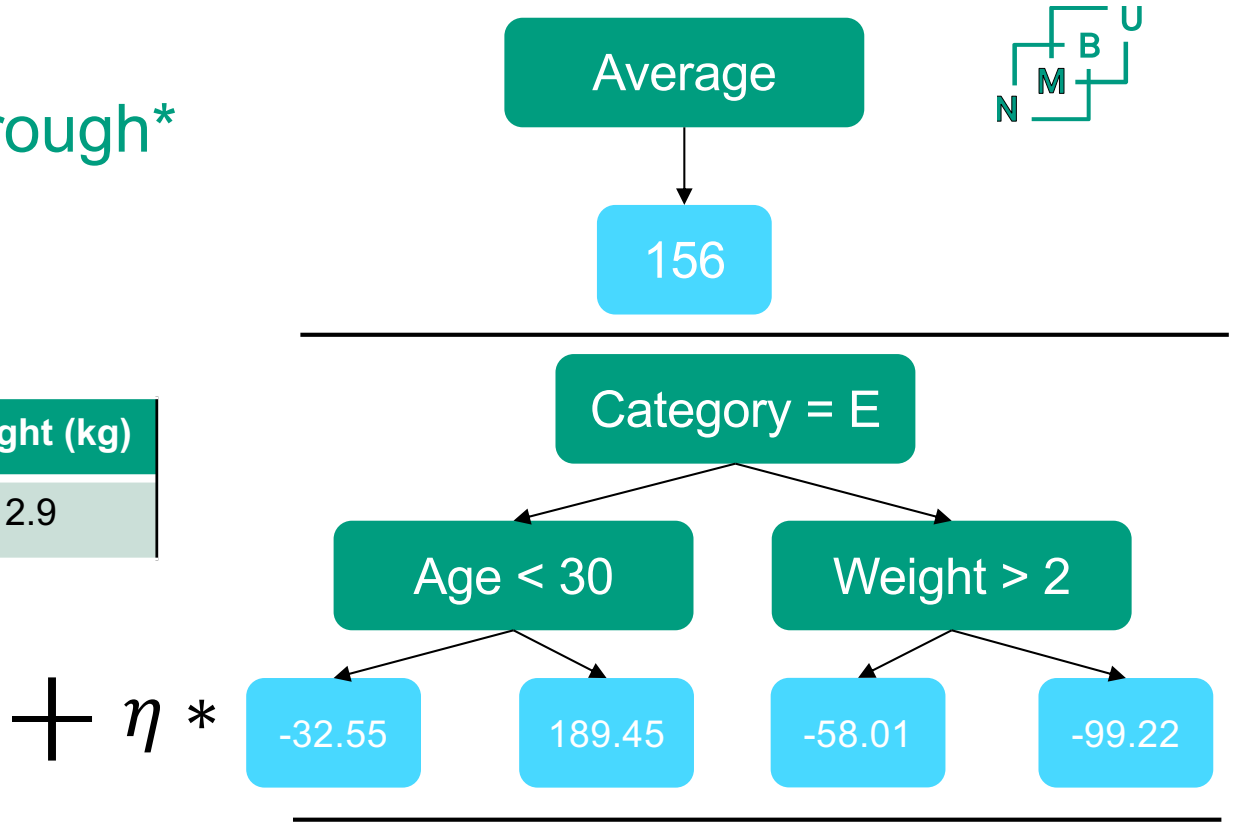
We repeat the same steps with Tree 2 .. Tree *n*

**https://www.datacamp.com/tutorial/guide-to-the-gradient-boosting-algorithm*

Step-by-step walk through*

- New prediction

Age	Category	Weight (kg)
30	Electronics	2.9



Predicted value= $156 + (0.1 * (-58.01)) + \ldots\ldots$

General rule:

Final prediction = average + η·Tree1 + η·Tree2 + η·Tree3 + ... + η·TreeN

*https://www.datacamp.com/tutorial/guide-to-the-gradient-boosting-algorithm

Gradient Boosting – algorithmic overview*

Gradient Boosting Algorithm

1. Initialize model with a constant value:

$$F_0(x) = \underset{\gamma}{argmin} \sum_{i=1}^{n} L(y_i, \gamma)$$

2. for $m = 1$ *to* M:

2-1. Compute residuals $r_{im} = -\left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)}\right]_{F(x)=F_{m-1}(x)}$ $for\ i = 1,\ldots,n$

2-2. Train regression tree with features x against r and create terminal node reasions R_{jm} for $j = 1,\ldots,J_m$

2-3. Compute $\gamma_{jm} = \underset{\gamma}{argmin} \sum_{x_i \in R_{jm}} L(y_i, F_{m-1}(x_i) + \gamma)$ $for\ j = 1,\ldots,J_m$

2-4. Update the model:

$$F_m(x) = F_{m-1}(x) + \nu \sum_{j=1}^{J_m} \gamma_{jm} 1(x \in R_{jm})$$

Source: adapted from Wikipedia and Friedman's paper

1. Initial prediction -> search for a value that minimizes the loss (mean)
2. Loop through *M* trees
   I. Calculate the derivative of the loss function with respect to the previous predictions
   II. Build a tree
   III. Minimize the average loss of the residuals on all the samples that belong in the terminal node R_{jm}
   IV. New prediction from the combined models

*https://towardsdatascience.com/all-you-need-to-know-about-gradient-boosting-algorithm-part-1-regression-2520a34a502

Gradient Boosting – Hyperparameters

Key Hyperparameters

Learning Rate
- Controls how much each tree influences the final model
- Smaller => more trees, better generalization
- Lower rate helps reduce overfitting

Number of Trees (n_estimators)
- Total number of boosting rounds
- More trees => better learning, but higher risk of overfitting
- Use with **early stopping** + **low learning rate**

Max Depth
- Tree complexity; deeper => more expressive but riskier
- Recommended: 3–10 for balance

Min Samples per Leaf
- Minimum number of data points to create a leaf
- Higher value prevents overfitting, reduces sensitivity to noise

Subsampling Rate
- Fraction of the rows used to train each tree
- <1 (e.g., 0.7) => faster training, reduces variance
- Helps with generalization

Feature Sampling Rate
- Fraction of the features used per tree
- Useful for high-dimensional datasets
- Recommended: 0.5–1 to reduce overfitting

Gradient Boosting – Hyperparameters

Key Hyperparameters

Learning Rate
- Controls how much each tree influences the final model
- Smaller = more trees, better generalization
- Lower rate helps reduce overfitting

Number of Trees (n_estimators)
- Total number of boosting rounds
- More trees = better learning, but higher risk of overfitting
- Use with **early stopping** + **low learning rate**

Max Depth
- Tree complexity; deeper = more expressive but riskier
- Recommended: 3–10 for balance

Min Samples per Leaf
- Minimum data points to create a leaf
- Higher value = prevents overfitting, reduces sensitivity to noise

Subsampling Rate
- Fraction of rows used to train each tree
- <1 (e.g., 0.7) = faster training, reduces variance
- Helps with generalization

Feature Sampling Rate
- Fraction of features used per tree
- Useful for high-dimensional datasets
- Recommended: 0.5–1 to reduce overfitting

In Sklearn:

```
from sklearn.ensemble import GradientBoostingRegressor

model_sklearn = GradientBoostingRegressor(
n_estimators=100,
learning_rate=0.1,
max_depth=3,
min_samples_split=10,
min_samples_leaf=5,
subsample=0.7,
random_state=42)
```

Gradient Boosting versus AdaBoost

- AdaBoost: Adjusts the weights of training instances to focus on the ones that **previous classifiers misclassified**.
- Gradient Boosting: Builds each tree on the **residuals of the previous tree's predictions**, effectively focusing on reducing the errors.

AdaBoost	Gradient Boosting
During each iteration in AdaBoost, the weights of incorrectly classified samples are increased, so that the next weak learner focuses more on these samples.	Gradient Boosting updates the weights by computing the negative gradient of the loss function with respect to the predicted output.
Calculates the weights for the weak classifiers in the final decision based on their accuracy.	Uses a learning rate to shrink the contribution of each tree to prevent overfitting and improve model robustness.
AdaBoost uses simple decision trees with one split known as decision stumps as weak learners.	Gradient Boosting can use a wide range of base learners, such as decision trees and linear models.
AdaBoost is more susceptible to noise and outliers in the data, as it assigns high weights to misclassified samples	Gradient Boosting is generally more robust, as it updates the weights based on the gradients, which are less sensitive to outliers.

The XGBoost

- XGBoost **is a more optimized and regularized version** of Gradient Boosting:
- **Key Differences:**

1. Regularization

  – Helps prevent overfitting (via lambda, gamma)

  – Gradient Boosting usually doesn't include this explicitly

2. Greedy tree construction with pruning

  – Uses the **gain score** to decide if a split improves the performance

  – **Prunes** branches if the improvement is too small (gain < gamma)

3. Similarity score for splitting

  – Quantifies how similar data points are in a node — helps make better splits

4. Efficiency

  – Much faster due to parallel processing, caching, and handling of sparse data

5. Formulaic update

  – Adds the new tree's **leaf values**, weighted by the learning rate

The XGBoost

- Aka eXtreme Gradient Boosting

1. Initial prediction
2. Compute the residuals
3. Build a weak learner
   I. (XG-unique*) decision/regression tree on the residuals (greedy algorithm logic)
   II. Similarity score is the evaluation metric for nodes
   III. Gain score is an evaluation criterion for trees
   IV. Pruning: If Gain score < γ, the branch is pruned
4. New Prediction = Previous Prediction + (Learning rate) x (New Prediction)

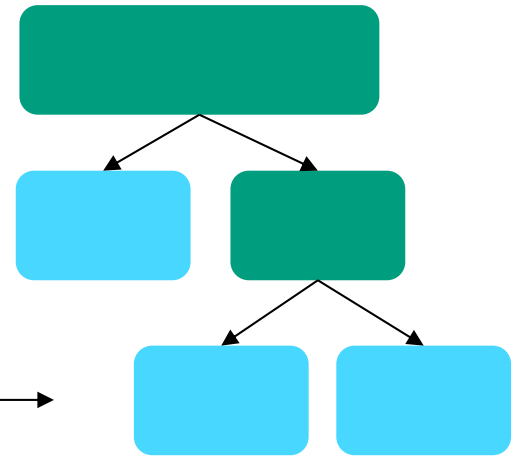
Repeat

* Regularized decision tree
Lambda: Regularization parameter which reduces the prediction´s **sensitivity to individual observations.**
Gamma: Specifies the minimum loss reduction required to make a further partition of the tree and controls the **actual structure of the trees**.

XG-unique Trees

$$Gain = Similarity_{left} + Similariy_{right} - Similarity_{root}$$

$$Similarity\ score = \frac{(sum\ of\ residuals)^2}{Number\ of\ residuals + \lambda}$$



The XGBoost

Advantage	Disadvantage
Performance (good track record on Kaggle competitions)	**Computational Complexity** (more computationally intensive steps)
Scalability (suitable for large datasets)	**Overfitting** (especially when trained on small datasets)
Customizability (a range of hyperparameters that can be adjusted, making it highly customizable)	**Hyperparameter tuning** (important to properly tune the parameters)
Interpretability (naturally provides feature importance through the gain and similarity score/cover)	**Memory requirements**

The XGBoost - Example

- XGBoost **is a more optimized and regularized version** of Gradient Boosting:

In Sklearn:

```
from xgboost import XGBRegressor

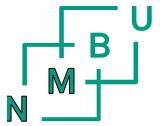
model_xgb =XGBRegressor( n_estimators=100,
learning_rate=0.1,
max_depth=3,
reg_lambda=1.0,
reg_alpha=0.1,
gamma=0.1,
min_child_weight=5,
subsample=0.7,
colsample_bytree=0.8,
random_state=42)
```

- n_estimators → number of trees
- learning_rate → how much each tree contributes (smaller => slower, safer learning)
- max_depth → limits tree complexity
- reg_lambda → L2 penalty (keeps predictions smooth, reduces overfitting)
- reg_alpha → L1 penalty (pushes some leaf weights toward zero)
- gamma → only split if improvement is big enough
- min_child_weight → require enough data in the leaves (avoid tiny regions)
- subsample=0.7 → use 70% of the data per tree (adds randomness)
- colsample_bytree=0.8 → use 80% of the features per tree (adds randomness)
- random_state=42 → ensures reproducibility

Notebook *Gradient_Boost_X-G__Boost.ipynb*

Summary

Bagging versus Boosting



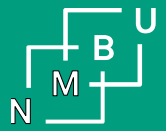
- **Bagging**
  - A parallel ensemble of classifiers, aiming to improve classification performance by decreasing model variance.
  - Especially useful for improving accuracy of unstable models and prevent against overfitting. Works as averaging across "overfitted" models.
  - Can degrade the performance of stable classifiers like k-NN.
  - Example: Random forests

- **Boosting**
  - A sequential ensemble of classifiers (the next classifier emphasizes errors made by the previous one), aiming at decreasing model bias.
  - Combines underfitted models.
    - The individual classifiers must be at least better than random guessing (but not necessarily "too good").
  - Examples: Gradient boosting, AdaBoost

Summary

- **Majority voting:**
  - (Weighted) consensus from many classifiers
  - Also used in multiclass classification problems

- **Ensemble learning is:**
  - computationally more time consuming,
  - prone to overfitting,
  - but capable of improving on other classifiers (reducing bias and variance).


U
B
M
N